

Modelli di processi SW

Trentin Tommaso

December 27, 2025

Contents

1	Processo vs Modello di processo	2
2	Modello si processo SW: Punti di vista	2
3	Descrizione del processo SW	2
4	Modello a Cascata (Waterfall)	2
4.1	Modello a cascata con retroazione	3
4.2	Modello a V	3
5	Modelli Evolutivi - Sviluppo Incrementale	3
6	Modelli Evolutivi - Consegna Incrementale	3
6.1	Consegna VS Sviluppo Incrementale	4
6.2	Consegna e Sviluppo Incrementale	4
7	Modelli Evolutivi - Prototipale	4
7.1	Modello Prototipale - Back-To-Back Testing	4
8	Modello Orientato al Riuso	5
9	Modello Trasformatzionale	5

1 Processo vs Modello di processo

Non esiste modello univale → ciascuna organizzazione usa proprio processo (possibilità di usare processi diversi per prodotti diversi).

Ogni processo comprende attività, sotto-attività, prodotti e persone con responsabilità.

Modello di processo SW: descrizione semplificata/astratta del processo SW, ci si riferisce a questi modelli con diversi alias (modelli di processo sw / ciclo di vita del sw / paradigmi di processo).

Descrizioni astratte di alto livello di processi sw che possono essere utilizzate per spiegare diversi approcci allo sviluppo sw.

2 Modello si processo SW: Punti di vista

Ogni processo ha diverse descrizioni possibili:

- Architetturale → descrive sequenza di attività senza fornire dettgli su specifiche attività;
- Data-flow → evidenzia trasformazioni dei dati operate da attività del processo;
- Role/action → ruoli delle persone coinvolte nel processo e relative responsabilità;

3 Descrizione del processo SW

Descrizione del processo include:

- **Attività** da svolgere e loro ordine;
- **Prodotti** risultanti da ciascuna attività;
- **Ruoli** (persone coinvolte e loro responsabilità)
- **Pre/Post condizioni** che devono verificarsi prima/dopo un'attività del processo sia svolta. Stabiliscono criteri di transizione per progredire da uno stadio del processo al successivo.

4 Modello a Cascata (Waterfall)

- Processo **plan-driven** in cui ciascuna fase segue la successiva, vanno pianificate tutte le attività di processo prima di iniziare sviluppo del SW. Gli output di una fase sono gli input della successiva;
- Processo **document-centric** guidato da produzione di documentazione, risultato di una fase è costituito da 1+ documenti approvati e la fase successiva non dovrebbe partire prima della fine della precedente e del completamento/approvazione dei suoi documenti;
- Processo **rigido** in cui i prodotti di una fase sono congelati (non modificabili se non innescando processo formale e sistematico di modifica), fine di ogni fase è una *milestone* del processo. Definizione precisa di milestone e output è importante per misurare progresso di un progetto;
- Processo **monolitico** in cui cliente vede il SW solo a completamento di tutte le fasi, in caso di errori nei requisiti questi vengono rilevati solamente alla fine con costi elevati. Processi non sono lineari ma ciascuna fase scambia feedback con le altre.

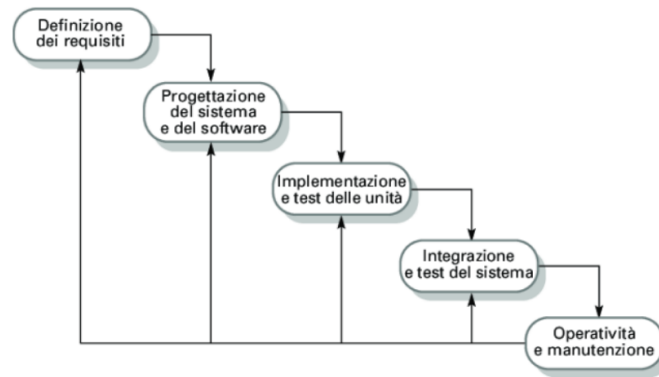
Vantaggi

- Fasi ben definite;
- Output di ciascuna fase precisamente individuati.

Svantaggi

- Richiede conoscenza immediata / stabilita dei requisiti (difficile in quanto spesso poco chiari anche al cliente)
- Sviluppo di eccessiva documentazione, presso non richiesta;
- Poca flessibilità (difficile gestire necessità di modifiche che emergono durante esecuzione del processo).

Adatto solo se requisiti sono ben chiari fin dall'inizio ed è difficile che cambino durante lo sviluppo, nella realtà pochi SW presentano requisiti stabili.



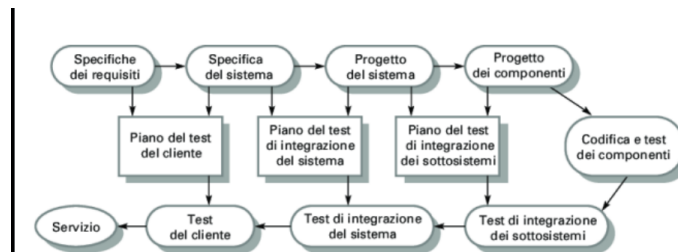
4.1 Modello a cascata con retroazione

Variazione del modello a cascata con inserimento di **retroazione**, ossia si verifica correttezza del risultato dell'attività con possibilità di ripetere attività precedente. Introduce dei **feedback** in ogni fase, in modo da rilevare errori prima della fase successiva, ad ogni feedback corrisponde una **potenziale modifica dei documenti** dello stadio precedente.

- *Mitiga* monoliticità del modello tradizionale (non necessario attendere termine del processo per modificare il prodotto);
- Non completamente flessibile rispetto a cambiamenti (possono avvenire in qualunque momento del processo)
- Utile quando si prevede che sistema sarà poco soggetto a cambiamenti.

4.2 Modello a V

Estensione del modello a cascata in cui attività del ramo superiore (**progetto**) sono collegate a quelle del ramo inferiore (**V&V**), in ogni fase del progetto il team definisce il corrispondente piano di test della fase VV (guidato da un'insieme di piani di test e eseguito da team indipendente da quello di sviluppo), se si trova un errore nella fase di VV **si rieseguono fasi di processo correlate**. Modello che può anticipare validazione dei requisiti da parte del cliente e garantire rilevamento precoce di errori.



5 Modelli Evolutivi - Sviluppo Incrementale

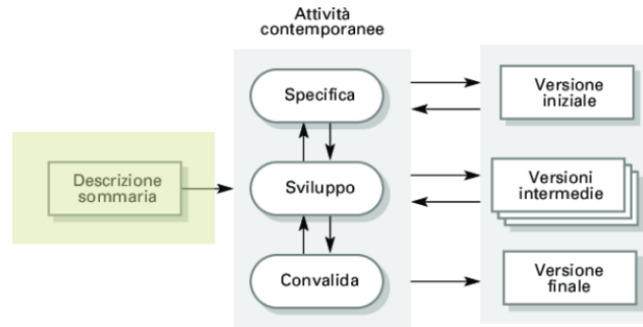
Idea: sviluppare un'implementazione iniziale (partendo da pochi requisiti chiari o da una descrizione sommaria dei requisiti), esporla e perfezionarla attraverso versioni successive (**incrementi**) fino ad ottenere il sistema richiesto. Attività di specifica, sviluppo e convalida sono intrecciate anziché separate (feedback veloci tra varie attività).

- Versione iniziale che implementa requisiti fondamentali viene esposta a utenti (clienti o loro proxy), feedback ottenibile più facilmente su una demo piuttosto che su documenti.
- Funzionalità / requisiti ulteriori vengono implementati in versioni intermedie successive (*solitamente* non mostrate al cliente)
- Ultimo incremento è la versione finale del sistema che viene rilasciata al cliente (corrisponde comunque a un'evoluzione della versione iniziale).

6 Modelli Evolutivi - Consegna Incrementale

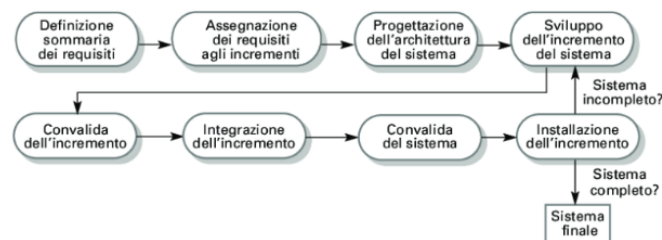
Sistema non viene consegnato direttamente alla sua forma finale alla fine del progetto, alcuni degli incrementi sviluppati sono consegnati ai clienti e installati nel loro ambiente operativo:

- **Vantaggio:** cliente usa incremento nell'ambiente operativo reale (feedback più realistico);



- **Limitazione:** utenti devono avere tempo sufficiente per sperimentare nuovi incrementi.

Idea: ogni incremento rilascia parte delle funzionalità richieste, ai requisiti utente vengono assegnati livelli di priorità (requisiti a priorità maggiore vengono rilasciati per primi). I requisiti di un incremento sono congelati dopo che tale incremento è stato consegnato, mentre gli altri possono evolvere. **Funzionalità comuni a più processi** dovrebbero essere individuate tempestivamente e **implementate all'inizio del processo**



6.1 Consegna VS Sviluppo Incrementale

Sviluppo incrementale: valutazione della prima versione effettuata da un proxy degli utenti finali in ambiente operativo diverso da quello target, *versioni intermedie solitamente non rilasciate al cliente.*

Consegna incrementale: valutazione più realistica e rappresentativa dell'utilizzo reale del SW in quanto ciascun incremento può essere *rilasciato agli utenti finali nell'ambiente operativo target.*

6.2 Consegna e Sviluppo Incrementale

Vantaggi:

- Rapido feedback del cliente su versione preliminare del SW invece che su documentazione;
- Possibilità di cambiare requisiti prima della consegna finale del prodotto, riducendo costi di modifica;
- Possibilità di consegnare a clienti versioni preliminari con funzionalità fondamentali già implementate;
- Primi incrementi possono essere utilizzati per dedurre requisiti per incrementi successivi;
- Funzionalità con priorità più elevata sono testate più approfonditamente.

Problemi

- Mancanza visibilità processo (anti-economico documentare ogni versione del sistema);
- Sistemi diventano spesso mal strutturati causa continui cambiamenti.

Applicabilità

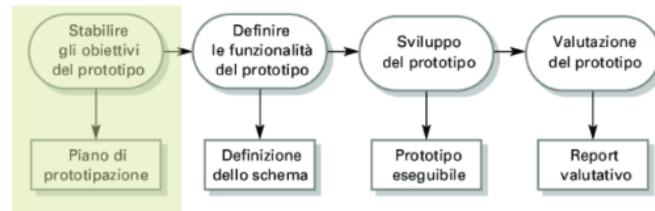
- Componenti di piccole / medie dimensioni (interfaccia utente);
- Sistemi destinati a vita breve;
- Sistemi i cui requisiti probabilmente varieranno durante lo sviluppo.

7 Modelli Evolutivi - Prototipale

Idea: **prototipo**, ossia versione iniziale di un intero sistema SW o parte di esso sviluppato rapidamente per contenere costi e sperimentare con il cliente prima della consegna (durante fasi iniziali del processo). Prototipo è **usa e getta** in quanto va scartato dopo sua validazione in quanto non è una buona base per sviluppare il sistema finale: pur

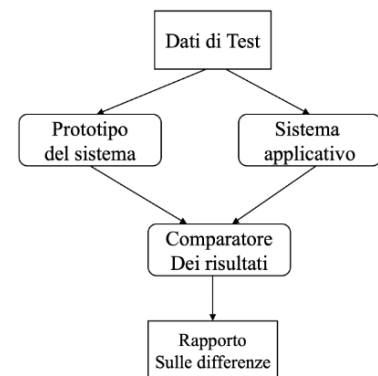
realizzando funzionalità richieste potrebbe non rispettare aspetti fondamentali (prestazioni o standard aziendali) o non essere documentato appropriatamente, inoltre rapidità di sviluppo e cambiamenti frequenti potrebbero deteriorarne la qualità.

- Obiettivi del prototipo devono essere stabiliti esplicitamente all'inizio del processo (altrimenti prototipazione potrebbe risultare inefficace);
- Non tutte le funzionalità del sistema finale devono essere incluse nel prototipo (per ridurre costi prototipazione);
- Necessaria formazione utenti su utilizzo dei prototipi (in quanto non rappresentativi del sistema reale);
- Prototipazione può essere combinata con altri cicli di vita classici (identificazione / validazione / raffinamento requisiti);
- Durante progettazione di un modello a cascata prototipazione può essere usata per valutare *opzioni alternative a progettazione*.



7.1 Modello Prototipale - Back-To-Back Testing

Prototipo può essere usato durante validazione per controllare che sistema sviluppato si comporti come modellato ne prototipo delle fasi iniziali del progetto.



8 Modello Orientato al Riuso

Approccio che tende al riuso di componenti SW riutilizzabili ma anche di interi sistemi (COTS - Commercial off the shelf). Sfrutta **framework di integrazione** per comporre componenti. **Componenti riutilizzabili e COTS** possono essere configurati per adattare loro comportamento a requisiti utente.

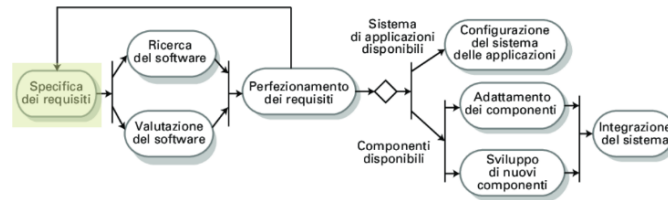
- Requisiti iniziali specificati in maniera non eccessivamente dettagliata;
- Vengono ricercati componenti e sistemi che possono fornire funzionalità richieste, candidati vengono valutati per vedere se soddisfano requisiti essenziali e se sono disponibili per essere utilizzati;
- Requisiti vengono perfezionati utilizzando informazioni sulle applicazioni / componenti trovate, la specifica viene aggiornata con i requisiti perfezionati;
- Se disponibile un sistema pronto all'uso che soddisfa i requisiti, si configura quello per creare il sistema nuovo, altrimenti si configurano i singoli componenti riutilizzabili e si integrano con componenti sviluppati ad hoc per creare il sistema finale.

Vantaggi:

- Riduce quantità di SW ex-novo da sviluppare;
- Costi e rischi ridotti;
- Maggiore velocità nella consegna

Svantaggi:

- Sistema potrebbe non soddisfare tutte le reali necessità degli utenti causa compromessi nei requisiti;
- Evoluzione dei componenti riutilizzati non è controllata direttamente.



9 Modello Trasformativale

Idea: specifiche definite mediante linguaggi formali (spec. algebriche / modelli di stato), utilizzo di tecniche di model checking per provare correttezza. Specifiche formali vengono trasformate automaticamente in codice (correttezza preservata / verifica ottenuta implicitamente)

- Requisiti specificati formalmente in fase di analisi (specifiche verificate automaticamente prima di essere trasformate da opportuni strumenti);
- Descrizione formale trasformata man mano in una meno astratta / più dettagliata (manualmente o mediante appositi strumenti);
- Processo di trasformazione può trarre vantaggio da componenti riutilizzabili.

Problemi

- Necessarie competenze in linguaggi formali;
- Alcune parti del sistema difficili da specificare formalmente;
- Difficoltà del cliente nel convalidare i requisiti.

Applicabilità

- Non adatto per sistemi di grandi dimensioni;
- Utilizzato per parti critiche (dimostrazione validità *by construction*).

